NORAIZ RANA

Task #26

Topic

Authentication, JWT

And Basics

Date:

20 August 2025

SUBMITTED TO:

APPVERSE TECHNOLOGIES

📓 Learning Report: Authentication (JWT & Firebase Auth Basics)

1. Introduction

Authentication is a key part of modern web applications because it ensures only authorized users can access protected resources. Two popular methods I learned are **JWT (JSON Web Token)** and **Firebase Authentication**. Both provide secure login systems but work differently:

- JWT is a **token-based authentication method** commonly used in backend APIs.
- Firebase Authentication is a **ready-to-use service by Google** that simplifies login via email, password, or third-party providers.

---

2. JSON Web Token (JWT)

Definition: JWT is a compact, URL-safe token format that allows secure data exchange between client and server.

Structure:

- **Header** (algorithm & type)
- **Payload** (user information & claims)
- **Signature** (verifies authenticity)

Flow:

1. User logs in with email/password.
2. Server verifies credentials and generates a JWT.
3. JWT is sent to the client and stored (e.g., in localStorage).
4. Client attaches JWT to API requests (`Authorization: Bearer <token>`).
5. Server verifies the token before giving access.

Advantages:

- Stateless (no session storage on server).
- Works across domains and services.
- Secure when used with HTTPS.

---

3. Firebase Authentication

Firebase Auth provides **ready-made authentication services** without building a backend system from scratch.

Features:

- Supports **Email/Password login**.
- Integrates with **Google, Facebook, GitHub, Apple** logins.
- Provides **anonymous login** for guest users.
- Manages **password resets** and **token refresh** automatically.

Basic Example (Email/Password):

```
import { getAuth,
createUserWithEmailAndPassword,
signInWithEmailAndPassword } from
"firebase/auth";

const auth = getAuth();

// Register user
createUserWithEmailAndPassword(auth,
"test@email.com", "password123")
  .then(user => console.log(user))
  .catch(err => console.error(err));

// Login user
signInWithEmailAndPassword(auth,
"test@email.com", "password123")
  .then(user => console.log("Logged in:", user))
```

```
  .catch(err => console.error(err));
```

---

4. Comparison: JWT vs Firebase Auth

Aspect	JWT	Firebase Auth
Setup	Requires backend setup	No backend needed
Scalability	Very scalable (stateless)	Scalable but tied to Firebase
Flexibility	Highly customizable	Easy but limited customization
Best For	APIs, microservices	Quick apps, mobile/web projects

---

5. My Learning Summary

From this topic, I learned:

- **JWT** is ideal for API authentication and distributed systems because it is lightweight and stateless.
- **Firebase Authentication** is beginner-friendly and faster to implement, especially for projects that don't require a custom backend.
- Both methods improve **security and user experience** by protecting sensitive routes and data.

Overall, understanding these two methods gave me a clear view of how authentication works in real-world applications and how to choose the right approach depending on the project.